viz-2

October 8, 2025

0.0.1 Bivariate Analysis

[2]:
```python
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
```

[30]:
```python
# Dataset: https://drive.google.com/file/d/1d03gCsDKZFuMyTrYwPRETUwOMgZpulHJ/
↪view
```

[3]:
```python
data = pd.read_csv("final_vg.csv")
data.head()
```

[3]:
```
    Rank                               Name Platform    Year         Genre  \
0   2061                               1942      NES  1985.0       Shooter
1   9137          ¡Shin Chan Flipa en colores!       DS  2007.0      Platform
2  14279  .hack: Sekai no Mukou ni + Versus      PS3  2012.0        Action
3   8359           .hack//G.U. Vol.1//Rebirth      PS2  2006.0  Role-Playing
4   7109          .hack//G.U. Vol.2//Reminisce      PS2  2006.0  Role-Playing

            Publisher  NA_Sales  EU_Sales  JP_Sales  Other_Sales  Global_Sales
0             Capcom  4.569217  3.033887  3.439352     1.991671     12.802935
1          505 Games  2.076955  1.493442  3.033887     0.394830      7.034163
2  Namco Bandai Games  1.145709  1.762339  1.493442     0.408693      4.982552
3  Namco Bandai Games  2.031986  1.389856  3.228043     0.394830      7.226880
4  Namco Bandai Games  2.792725  2.592054  1.440483     1.493442      8.363113
```

[4]:
```python
data.describe()
```

[4]:
```
               Rank          Year      NA_Sales      EU_Sales      JP_Sales  \
count  16652.000000  16381.000000  16652.000000  16652.000000  16652.000000
mean    8283.409620   2006.390513      2.752314      1.996875      2.499677
std     4794.471477      5.863261      1.327002      1.322972      1.164023
min        1.000000   1980.000000      0.140000      0.010000      0.000000
25%     4129.750000   2003.000000      1.781124      1.087977      1.781124
50%     8273.500000   2007.000000      2.697415      1.714664      2.480356
75%    12436.250000   2010.000000      3.677290      2.795123      3.176299
```

```
max    16600.000000  2020.000000     8.725452     8.367985     12.722984

        Other_Sales  Global_Sales
count  16652.000000  16652.000000
mean       1.151829      8.457873
std        1.054813      3.717756
min       -0.474276      0.240000
25%        0.394830      5.580341
50%        0.491870      7.536614
75%        1.781124     11.227334
max        7.358020     30.555862
```

[ ]:

```

```

[8]:

```
# Most popular games
```

[9]:

```
data['Name'].value_counts()
```

[9]:

```
Ice Hockey                                      41
Baseball                                        17
Need for Speed: Most Wanted                     12
Ratatouille                                      9
FIFA 14                                          9
                                                ..
Indy 500                                         1
Indy Racing 2000                                 1
Indycar Series 2005                              1
inFAMOUS                                         1
Zyuden Sentai Kyoryuger: Game de Gaburincho!!    1
Name: Name, Length: 11493, dtype: int64
```

[15]:

```
ih = data[data['Name'] == 'Ice Hockey']
```

[17]:

```
len(ih)
```

[17]:

```
41
```

[16]:

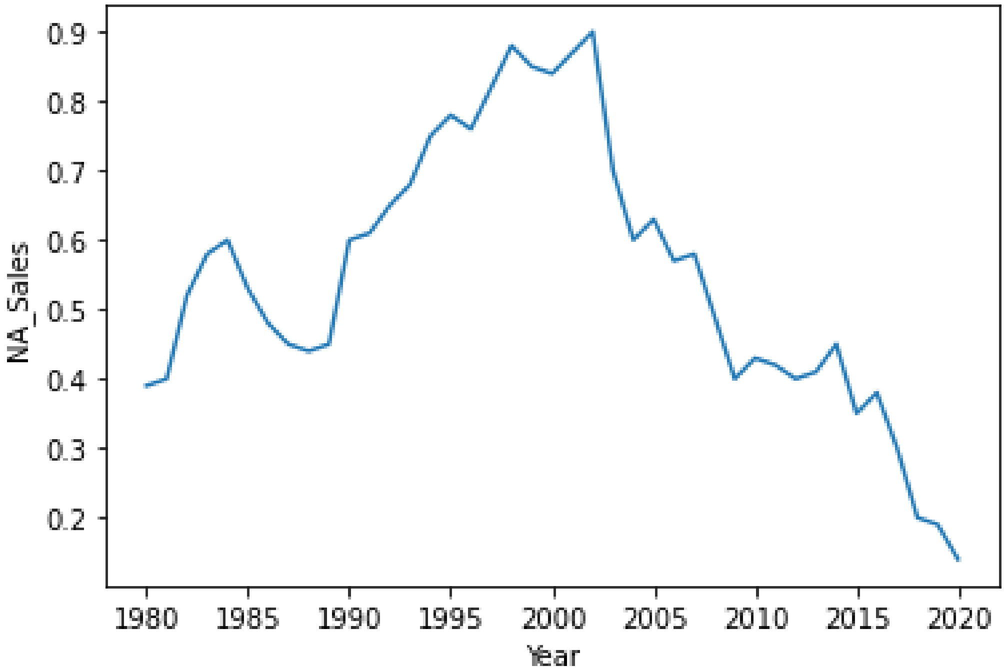
```
baseball = data[data['Name'] == 'Baseball']
```

[18]:

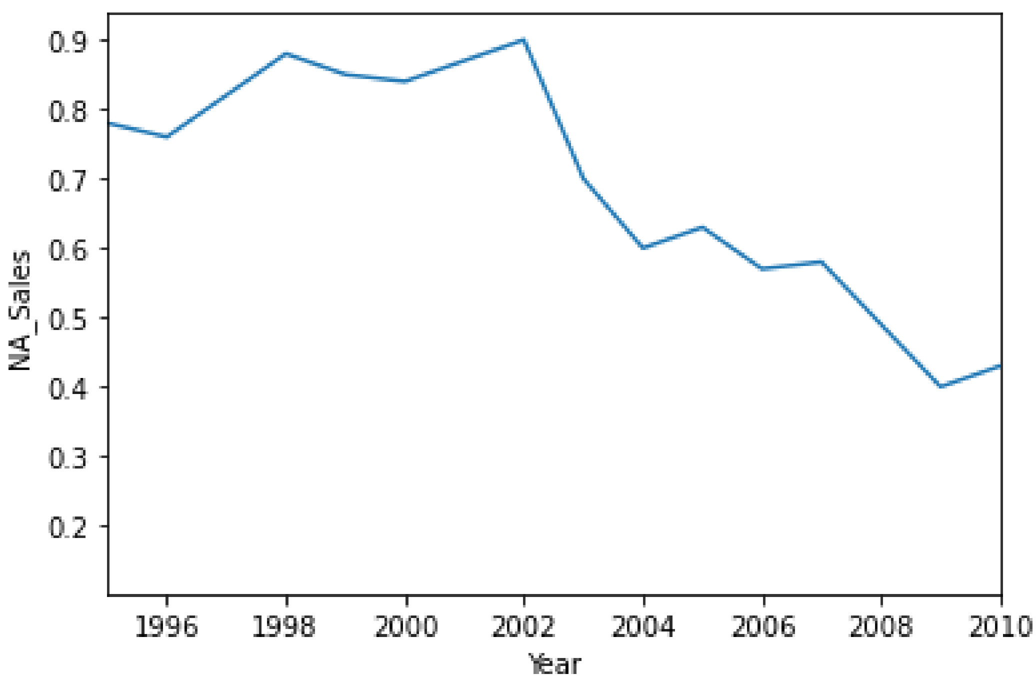
```
# For NA sales trend of ice hockey games year wise?
```

[29]:

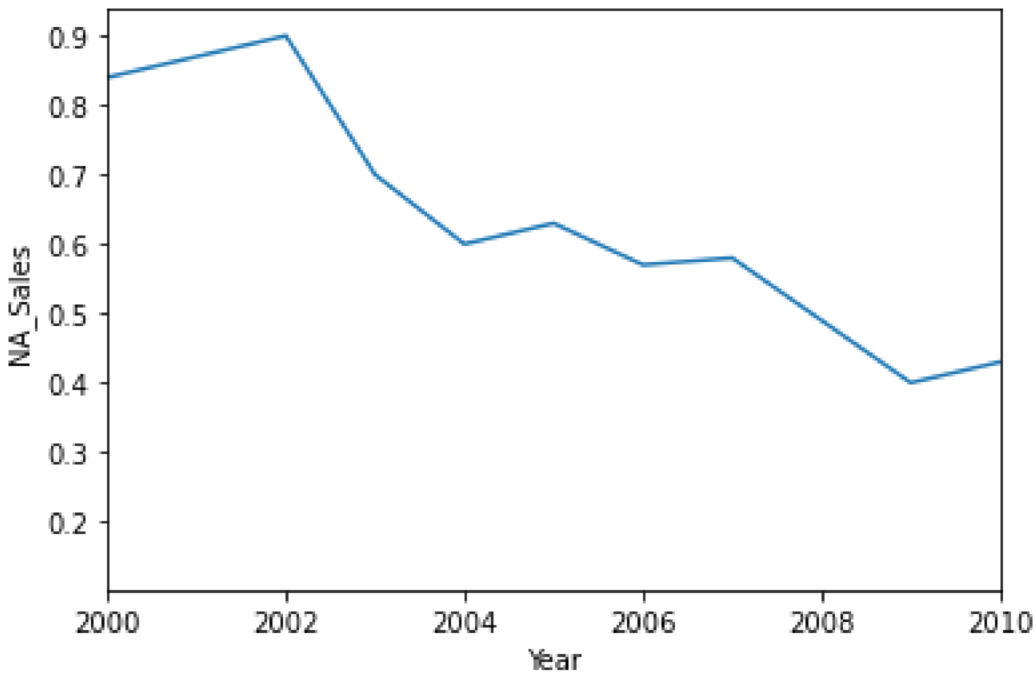
```
sns.lineplot(data = ih, x = 'Year', y = 'NA_Sales') # Bivariate
plt.show()
```



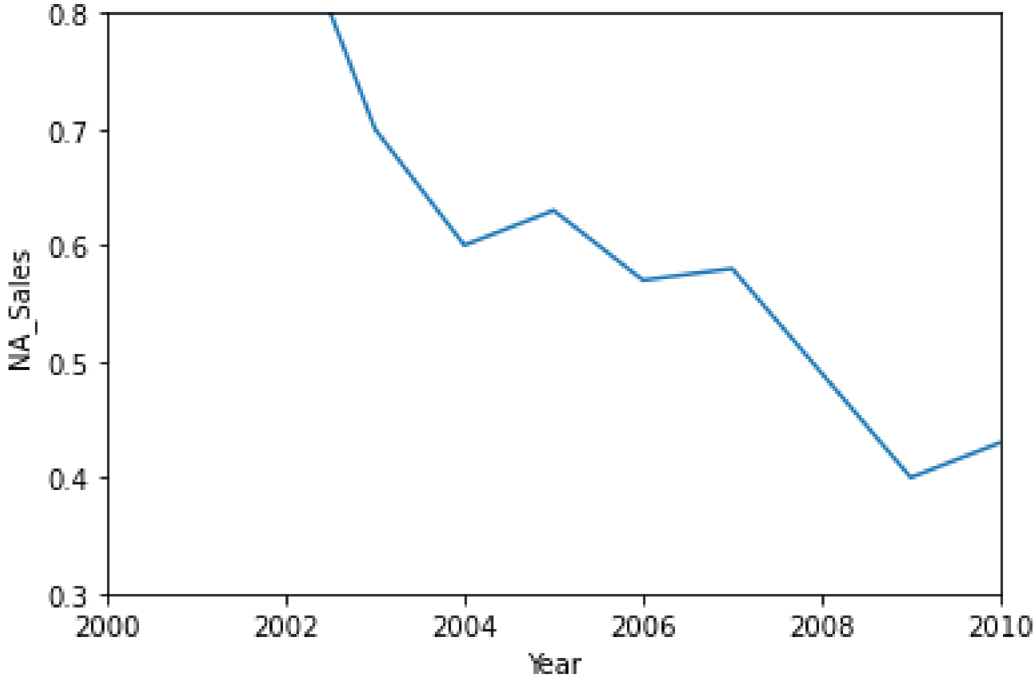
[23]:
```
sns.lineplot(data = ih, x = 'Year', y = 'NA_Sales')
plt.xlim(1995, 2010)
plt.show()
```



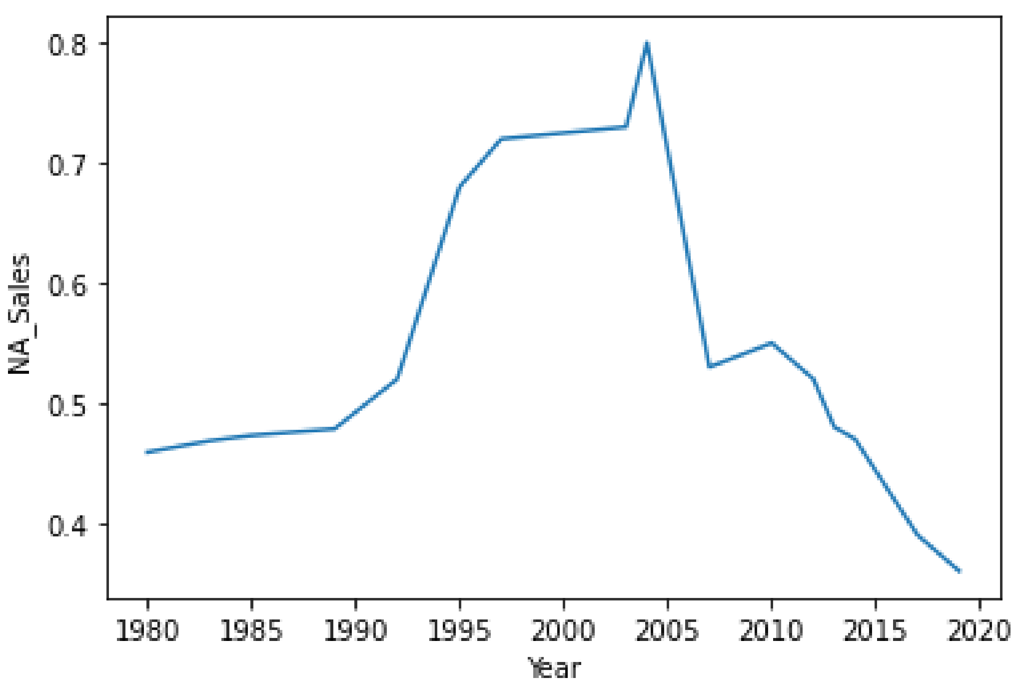
[24]:
```
sns.lineplot(data = ih, x = 'Year', y = 'NA_Sales')
plt.xlim(2000, 2010)
plt.show()
```



[27]:
```
sns.lineplot(data = ih, x = 'Year', y = 'NA_Sales')
plt.xlim(2000, 2010)
plt.ylim(0.3, 0.8)
plt.show()
```

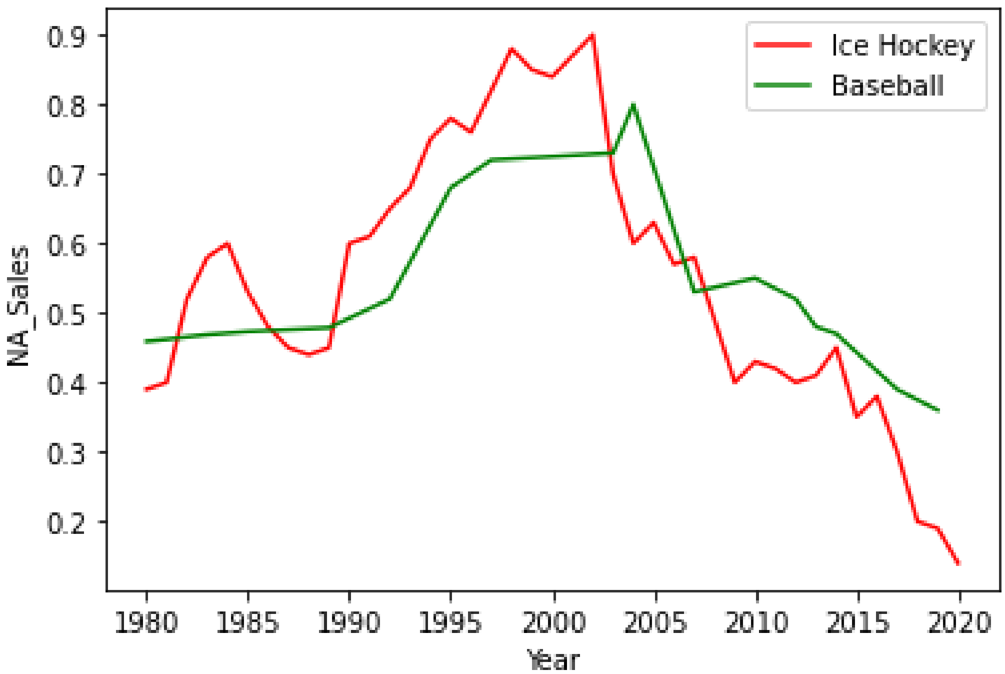


[28]:
```
sns.lineplot(data = baseball, x = 'Year', y = 'NA_Sales')
plt.show()
```



[31]:
```
# Lineplot together -> plotting of 2 games in a single plot:
```

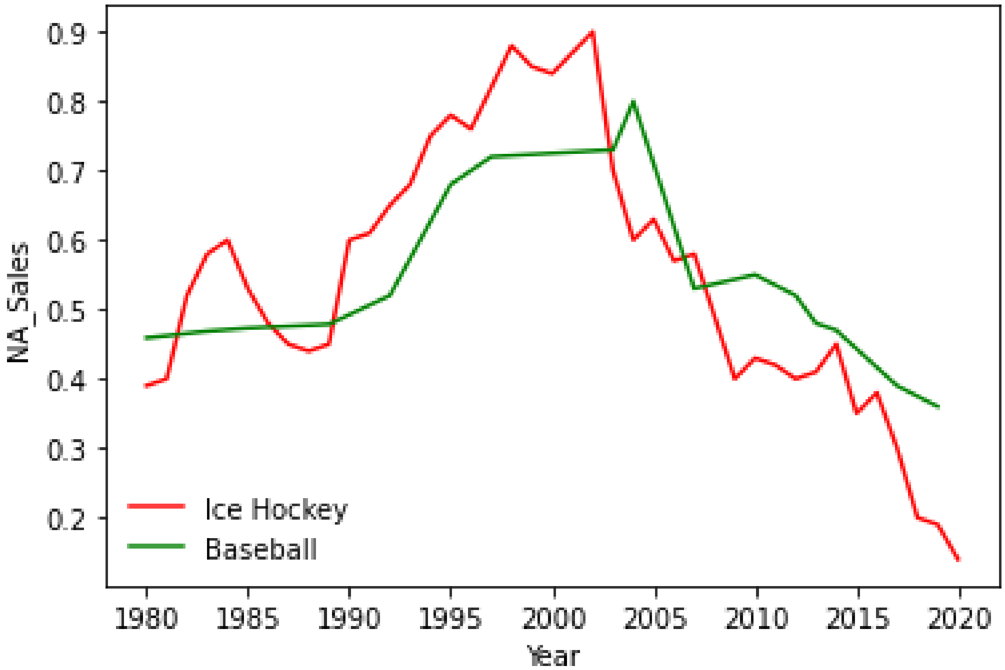
[33]:
```
sns.lineplot(data = ih, x = 'Year', y = 'NA_Sales', color = 'red', label = 'Ice␣
↪Hockey')
sns.lineplot(data = baseball, x = 'Year', y = 'NA_Sales', color = 'green',␣
↪label='Baseball')
plt.legend()
plt.show()
```



[36]:
```
# try documentation once
```

[37]:
```
# CN
```

[35]:
```
sns.lineplot(data = ih, x = 'Year', y = 'NA_Sales', color = 'red', label = 'Ice␣
↪Hockey')
sns.lineplot(data = baseball, x = 'Year', y = 'NA_Sales', color = 'green',␣
↪label='Baseball')
plt.legend(loc = 'lower left', frameon=False)
plt.show()
```



[ ]:

```

```

[38]:

```
# NN
```

[40]:

```
# Find out if there is a correlation between Rank and Global Sales of that game
```

[39]:

```
data.head()
```

[39]:

```
    Rank                            Name Platform    Year         Genre  \
0   2061                            1942      NES  1985.0       Shooter
1   9137        ¡Shin Chan Flipa en colores!       DS  2007.0      Platform
2  14279  .hack: Sekai no Mukou ni + Versus      PS3  2012.0        Action
3   8359          .hack//G.U. Vol.1//Rebirth      PS2  2006.0  Role-Playing
4   7109        .hack//G.U. Vol.2//Reminisce      PS2  2006.0  Role-Playing

            Publisher  NA_Sales  EU_Sales  JP_Sales  Other_Sales  Global_Sales
0              Capcom  4.569217  3.033887  3.439352     1.991671     12.802935
1           505 Games  2.076955  1.493442  3.033887     0.394830      7.034163
2  Namco Bandai Games  1.145709  1.762339  1.493442     0.408693      4.982552
3  Namco Bandai Games  2.031986  1.389856  3.228043     0.394830      7.226880
4  Namco Bandai Games  2.792725  2.592054  1.440483     1.493442      8.363113
```

[42]:

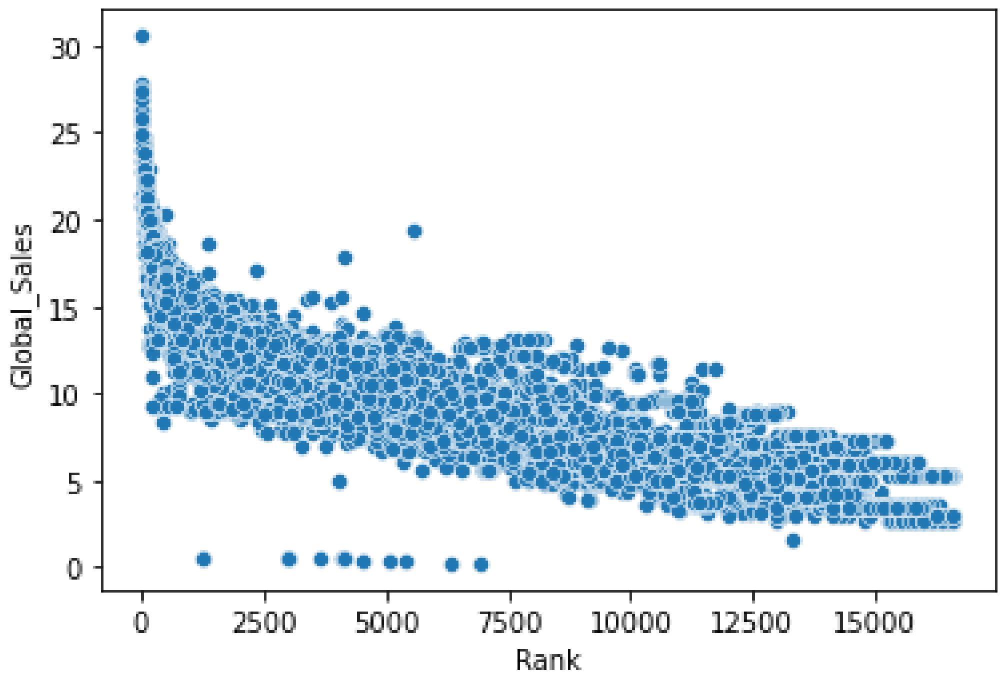
```
# sales vs rank: Scatter plot
```

[44]:

```
len(data)
```

[44]: 16652

[43]:
```python
sns.scatterplot(data = data, x='Rank', y='Global_Sales')
plt.show()
```



[ ]:

[45]:
```python
# CC
```

[48]:
```python
data.head()
```

[48]:
```
    Rank                              Name Platform    Year         Genre  \
0   2061                              1942      NES  1985.0       Shooter
1   9137         ¡Shin Chan Flipa en colores!       DS  2007.0      Platform
2  14279  .hack: Sekai no Mukou ni + Versus      PS3  2012.0        Action
3   8359          .hack//G.U. Vol.1//Rebirth      PS2  2006.0  Role-Playing
4   7109       .hack//G.U. Vol.2//Reminisce      PS2  2006.0  Role-Playing

            Publisher  NA_Sales  EU_Sales  JP_Sales  Other_Sales  Global_Sales
0              Capcom  4.569217  3.033887  3.439352     1.991671     12.802935
1           505 Games  2.076955  1.493442  3.033887     0.394830      7.034163
2  Namco Bandai Games  1.145709  1.762339  1.493442     0.408693      4.982552
3  Namco Bandai Games  2.031986  1.389856  3.228043     0.394830      7.226880
4  Namco Bandai Games  2.792725  2.592054  1.440483     1.493442      8.363113
```

[47]:
```python
# Get data of only those games which falls in each of the following categories:
# Should belong to top 3 publishers
# Should belong to top 3 Genres
# Should belong to top 3 platforms
```

[49]:
```python
# Top 3 publishers:
```

[116]:
```python
data['Publisher'].value_counts().head(3)
```

[116]:
```
Electronic Arts       1351
Activision            1014
Namco Bandai Games     932
Name: Publisher, dtype: int64
```

[108]:
```python
data['Publisher'].value_counts().head(3).index
```

[108]:
```
Index(['Electronic Arts', 'Activision', 'Namco Bandai Games'], dtype='object')
```

[53]:
```python
data['Publisher'].value_counts().index[:3]
```

[53]:
```
Index(['Electronic Arts', 'Activision', 'Namco Bandai Games'], dtype='object')
```

[114]:
```python
data['Publisher'].value_counts().head(3).index
```

[114]:
```
Index(['Electronic Arts', 'Activision', 'Namco Bandai Games'], dtype='object')
```

[95]:
```python
top3_pub = data['Publisher'].value_counts().index[:3]
```

[96]:
```python
top3_pub
```

[96]:
```
Index(['Electronic Arts', 'Activision', 'Namco Bandai Games'], dtype='object')
```

[97]:
```python
# Top 3 genre

top3_gen = data['Genre'].value_counts().index[:3]
```

[98]:
```python
top3_gen
```

[98]:
```
Index(['Action', 'Sports', 'Misc'], dtype='object')
```

[99]:
```python
# Top 3 platforms

top3_plat = data['Platform'].value_counts().index[:3]
```

[100]:
```python
print(top3_gen, top3_pub, top3_plat)
```

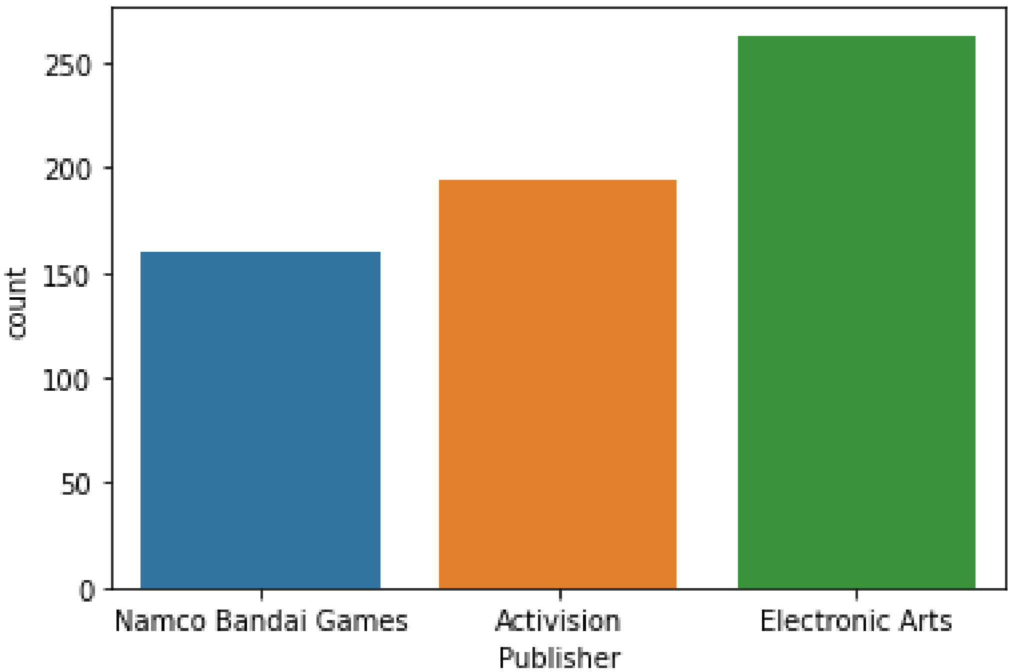
Index(['Action', 'Sports', 'Misc'], dtype='object') Index(['Electronic Arts', 'Activision', 'Namco Bandai Games'], dtype='object') Index(['DS', 'PS2', 'PS3'], dtype='object')

[105]:
```
top3_data = data[(data['Publisher'].isin(top3_pub)) &
                 (data['Platform'].isin(top3_plat)) &
                 (data['Genre'].isin(top3_gen))]
```

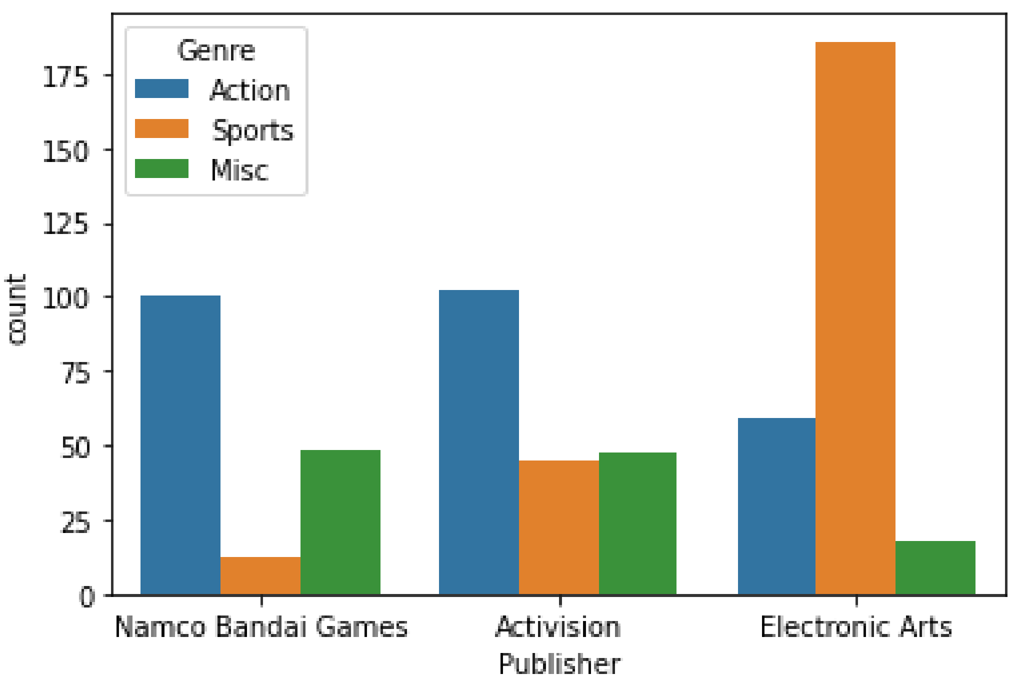
[115]:
```
top3_data.head()
```

[115]:
```
     Rank                              Name Platform    Year   Genre  \
2   14279  .hack: Sekai no Mukou ni + Versus      PS3  2012.0  Action
13   2742                     [Prototype 2]      PS3  2012.0  Action
16   1604                       [Prototype]      PS3  2009.0  Action
19   1741           007: Quantum of Solace      PS3  2008.0  Action
21   4501           007: Quantum of Solace      PS2  2008.0  Action

             Publisher  NA_Sales  EU_Sales  JP_Sales  Other_Sales  \
2   Namco Bandai Games  1.145709  1.762339  1.493442     0.408693
13          Activision  3.978349  3.727034  0.848807     2.792725
16          Activision  4.569217  4.108402  1.187272     3.339269
19          Activision  4.156030  4.346074  1.087977     3.390562
21          Activision  3.228043  2.738800  2.585598     3.652926

    Global_Sales
2       4.982552
13     11.447989
16     13.181205
19     12.980643
21     11.780257
```

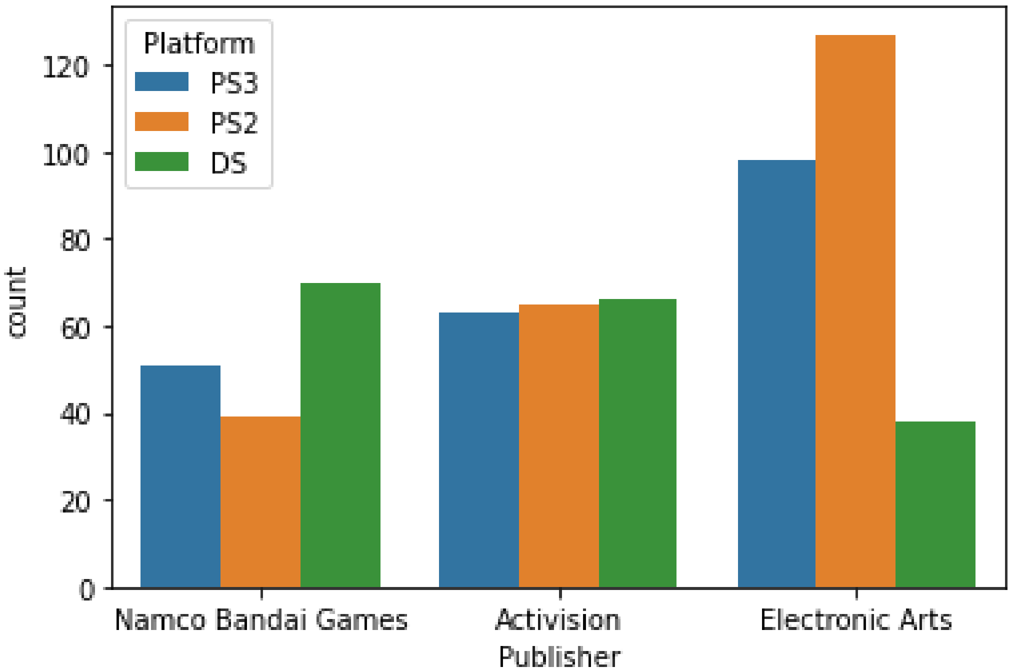
[118]:
```
sns.countplot(data = top3_data, x = 'Publisher') # Univariate
plt.show()
```



[119]:
```
sns.countplot(data = top3_data, x = 'Publisher', hue = 'Genre') # Univariate
plt.show()
```



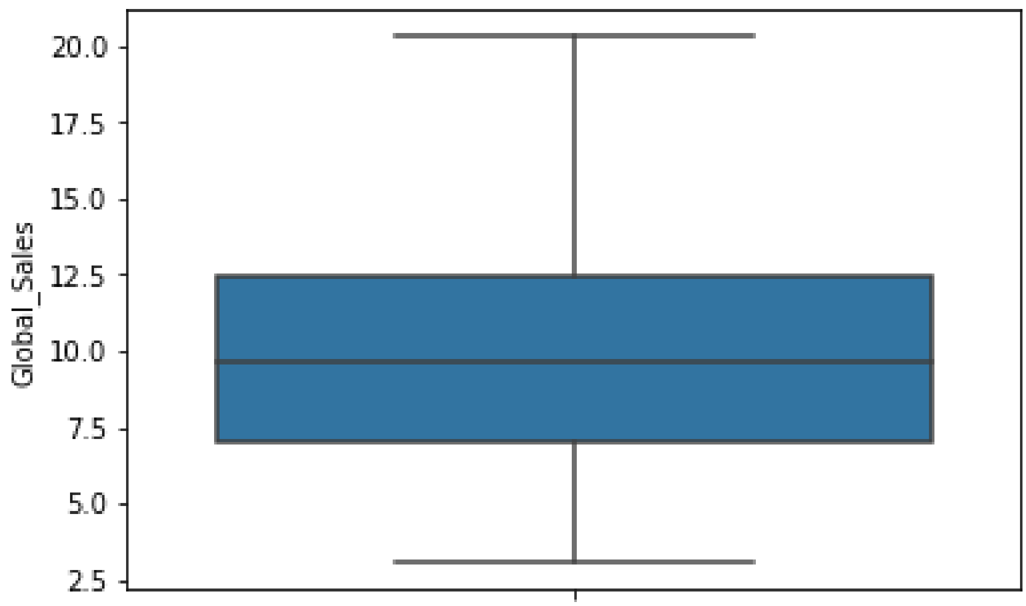
[121]:
```
sns.countplot(data = top3_data, x = 'Publisher', hue = 'Platform') # Univariate
plt.show()
```



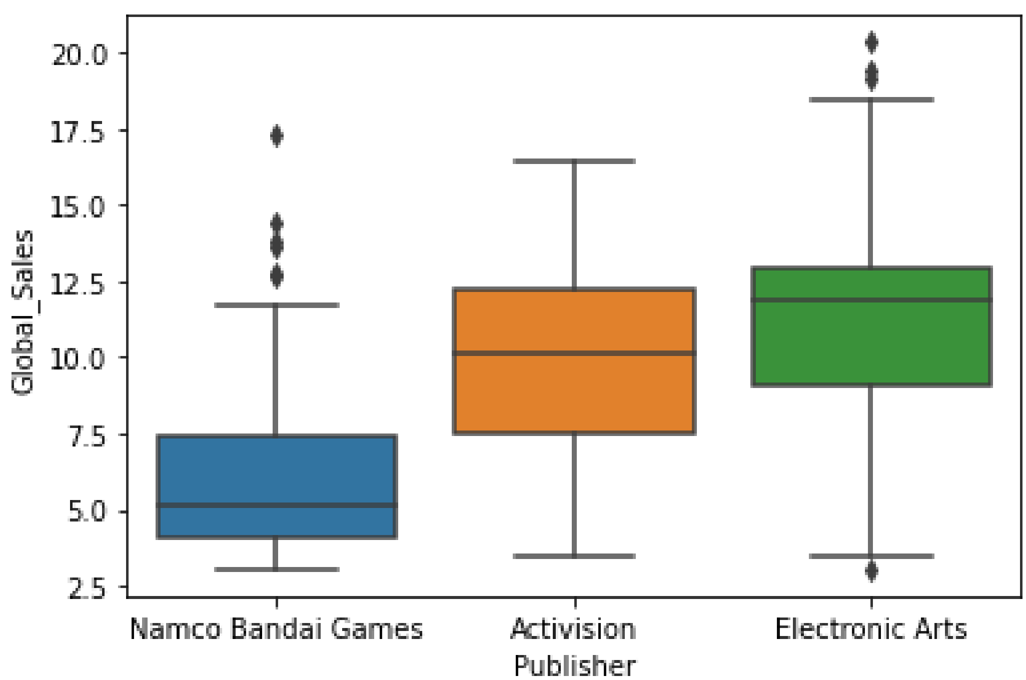
[120]:
```
# Conclusion:
```

[ ]:
```

```

[124]:
```
sns.boxplot(data = top3_data, y='Global_Sales') # univariate
plt.show()
```

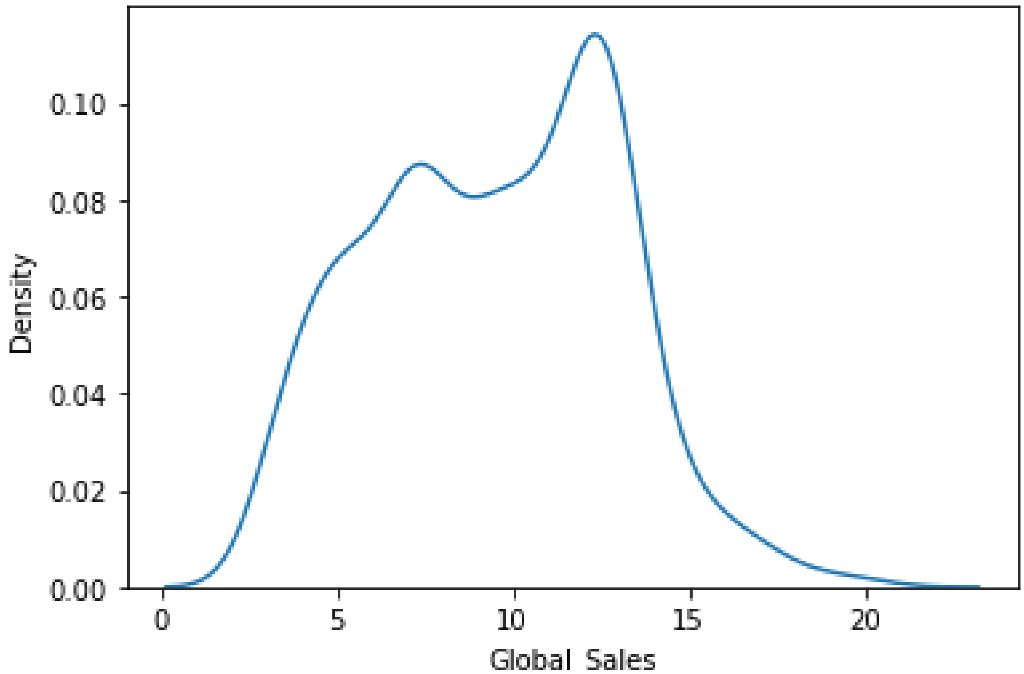


[125]:
```
sns.boxplot(data = top3_data, x = 'Publisher', y='Global_Sales') # bivariate
plt.show()
```

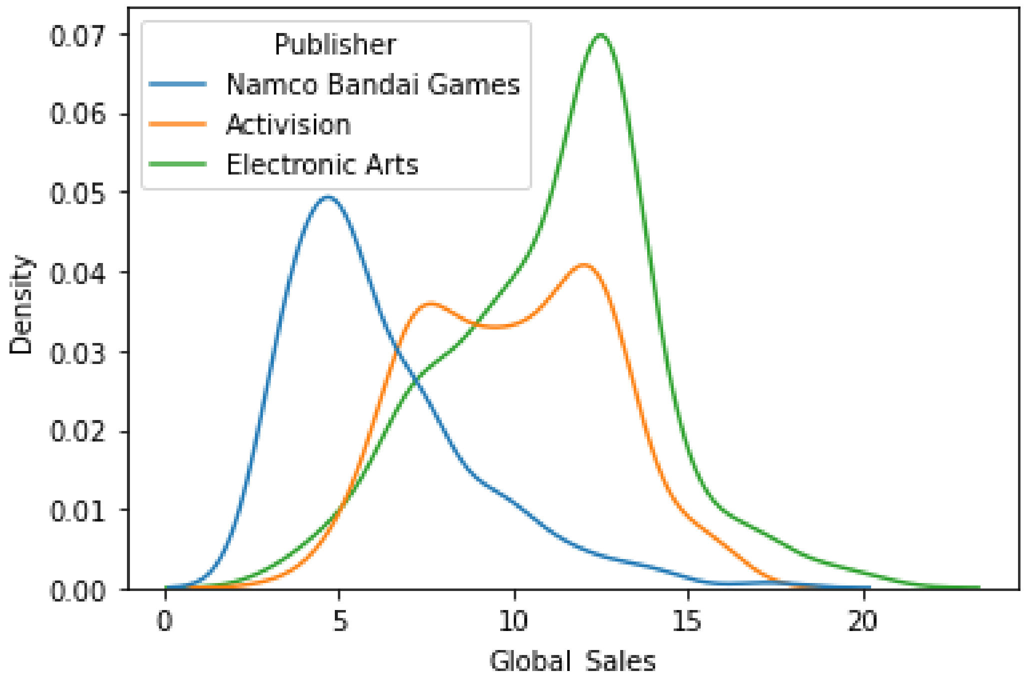


[126]:
```
# HW: Conclusion
```

[128]:
```
sns.kdeplot(data = top3_data, x = 'Global_Sales') # univariate
plt.show()
```



[129]:
```
sns.kdeplot(data = top3_data, x = 'Global_Sales', hue = 'Publisher') # bivariate
plt.show()
```



[ ]:

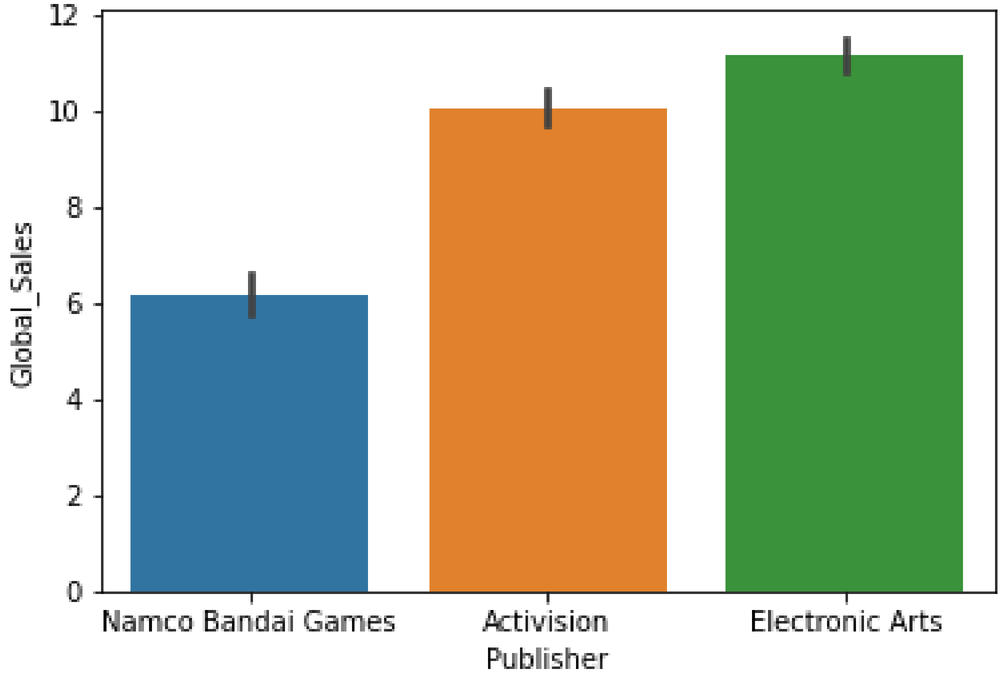
```

```

[130]:

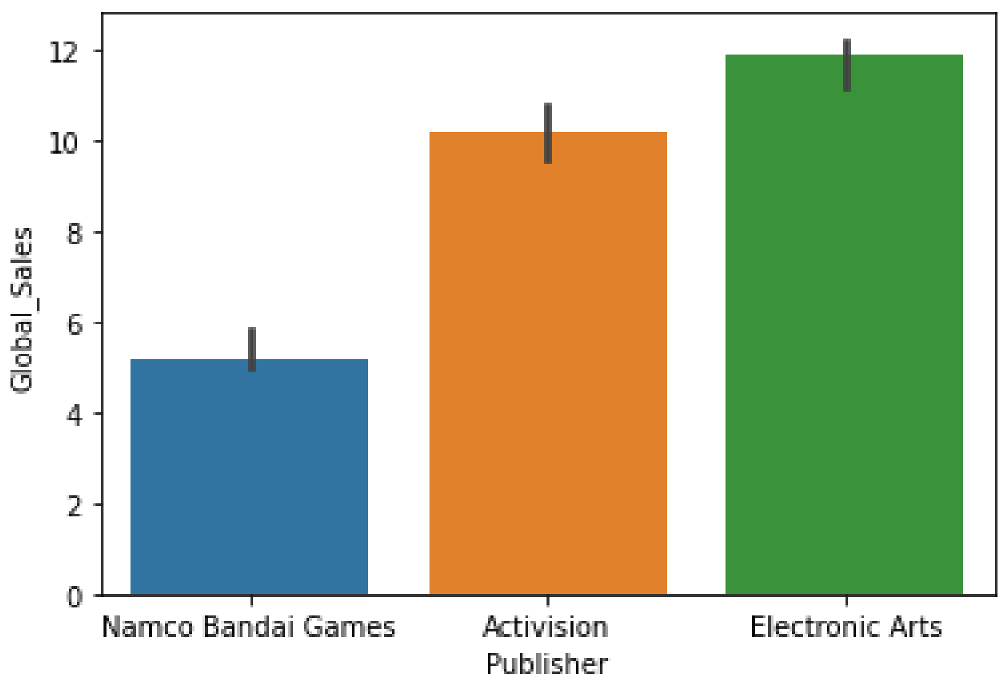
```
# For every publisher get avg global sales:
# Numerical vs Categorical
```

[132]:

```
sns.barplot(data = top3_data, x = 'Publisher', y = 'Global_Sales')
plt.show()
```



[137]:
```
sns.barplot(data = top3_data, x = 'Publisher', y = 'Global_Sales', estimator =␣
↪np.median)
plt.show()
```



[133]:
```
# Get mean value of each of these publisher's global sales through code?
```

[ ]:

[138]:
```
# Quiz:
```

- For the company "Toyota", we want to find which type of vehicle has made the maximum sales. Which plot we will prefer to use here?
- Ans: Bar plot
- Apple wanted to conduct an analysis and find the relationship between price and number of units sold for it's products. Which of the following plots would you prefer?
- Ans: Scatter plot

[139]:
```
top3_data.describe()
```

[139]:
```
                Rank         Year    NA_Sales    EU_Sales    JP_Sales  \
count     617.000000   611.000000  617.000000  617.000000  617.000000
mean     6532.186386  2008.036007    3.145065    2.292522    2.264623
std      4555.109759     3.496886    1.374738    1.416411    1.044837
min        83.000000  2000.000000    0.394830    0.394830    0.394830
25%      2609.000000  2006.000000    2.031986    1.087977    1.493442
50%      5552.000000  2008.000000    3.228043    1.894355    2.353444
75%     10041.000000  2010.000000    4.132499    3.390562    3.000569
max     16532.000000  2016.000000    6.449269    6.619388    5.363208

       Other_Sales  Global_Sales
count   617.000000    617.000000
mean      1.786518      9.512888
std       1.306538      3.474222
min       0.394830      3.046999
25%       0.394830      6.977851
50%       1.781124      9.675423
75%       2.697415     12.444357
max       5.900161     20.335571
```

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```